SISTEMA DE GESTIÓN DE SEMICONDUCTORES Y MECATRÓNICA

Arquitectura de Datos y Diagrama UML de Colecciones MongoDB

Modelado NoSQL de Documentos Raíz, Subdocumentos Embebidos (Embedding) y Relaciones Referenciadas (1:1 y 1:N) con django-mongodb-backend

Backend de Base de Datos: MongoDB Community / Atlas	**Framework Web:** Django 6.1 (Python 3.12+)
Conector Oficial: django-mongodb-backend	**Módulo / App:** miapp (Catálogo y POS Mecatrónica)

🏛️ Introducción y Principios de Modelado en MongoDB

El modelado de datos en MongoDB difiere fundamentalmente de los modelos relacionales (SQL). En lugar de normalizar datos a ultranza mediante docenas de tablas unidas por costosas operaciones de JOIN en tiempo de consulta, MongoDB y django-mongodb-backend permiten adoptar dos estrategias complementarias: la incrustación (Embedding) de subdocumentos y la referenciación (Referencing) mediante identificadores ObjectId.

En este sistema de comercio electrónico y punto de venta especializado en semiconductores y mecatrónica, la arquitectura fue diseñada meticulosamente considerando la atomicidad de las transacciones, la inmutabilidad histórica de las ventas y la contención de bloqueos durante picos de concurrencia.

- **1. Patrón de Documento Embebido (Embedded Documents):** Se utiliza para datos fuertemente ligados a una entidad padre que se leen juntos y que no tienen sentido sin ella (ej. Direccion de un usuario, PrecioEscalonado de un componente y DetalleVenta de una orden). Permite lecturas atómicas en 1 sola operación BSON sin ningún JOIN.
- **2. Patrón de Referencia Normalizada (References):** Se utiliza cuando las entidades crecen de forma independiente o son compartidas por múltiples consumidores (ej. Usuario y sus Órdenes, Categoría y Productos, Proveedores).
- **3. Patrón de Desacoplamiento de Inventario (Separated Collection):** Se aísla la colección 'inventory' del catálogo 'products' en una relación 1 a 1 referenciada. Esto evita que escrituras masivas de bloqueo de stock (reservas, checkout, ventas en POS) generen bloqueos o invalidación de cachés en el catálogo público de productos.

> ⚖️ **PRINCIPIO ARQUITECTÓNICO FUNDAMENTAL**
> Regla de Oro en MongoDB: 'Los datos que se acceden y consultan juntos, deben residir juntos'. Sin embargo, 'los datos con frecuencias de actualización radicalmente diferentes (lecturas de catálogo vs escrituras de stock) deben

> desacoplarse para garantizar escalabilidad'.

📊 Diagrama Gráfico UML de Clases y Colecciones

A continuación se presenta el diagrama UML de clases y colecciones que modela la base de datos completa. El diagrama distingue visualmente entre colecciones raíz independientes de MongoDB, subdocumentos embebidos (EmbeddedModel) y colecciones auxiliares o desacopladas, señalando con exactitud la cardinalidad (1:1 y 1:N) y el tipo de relación.

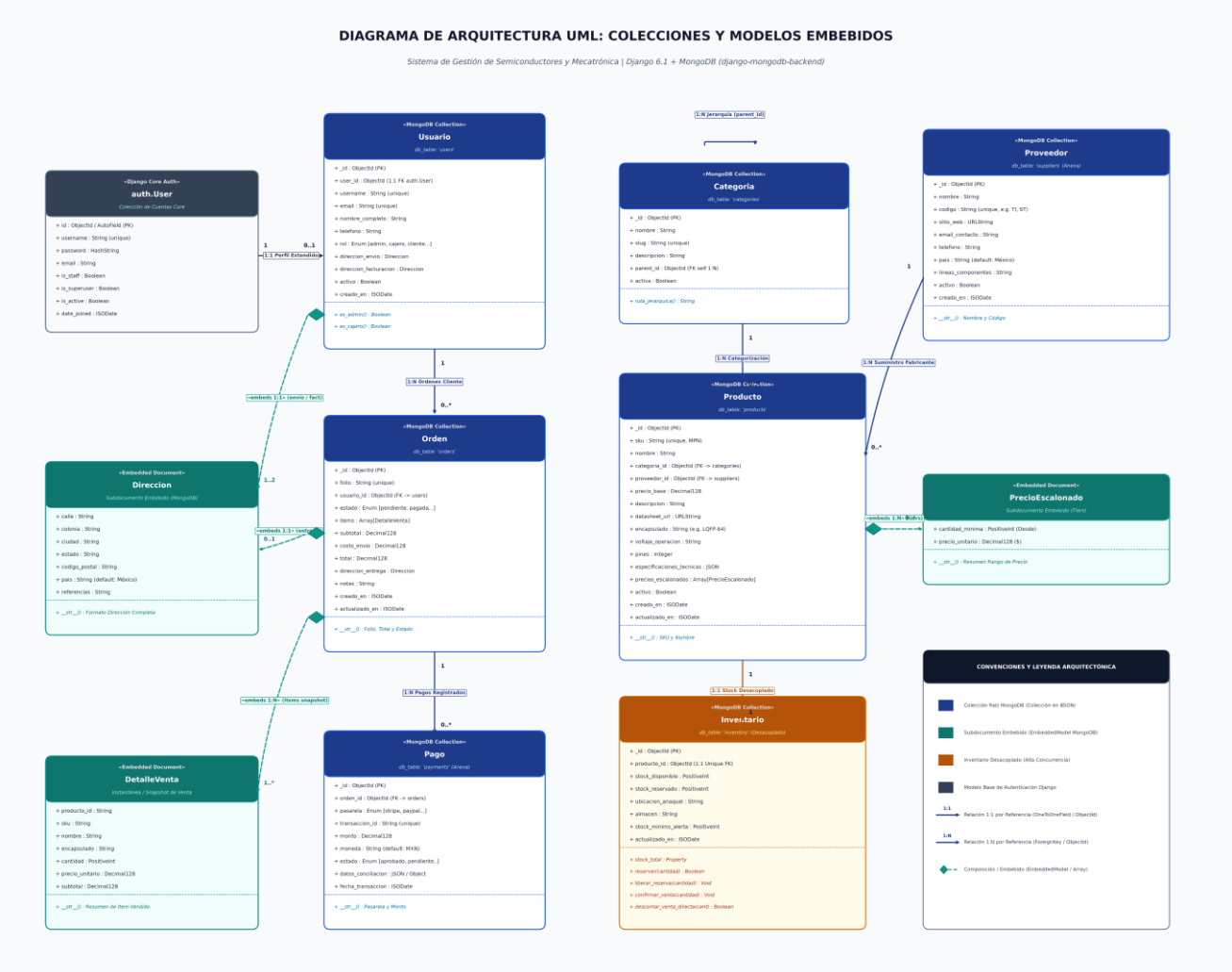


Figura 1: Diagrama de Arquitectura UML - Colecciones, Subdocumentos Embebidos y Relaciones NoSQL.

Guía de Notación y Convenciones del Diagrama

- **Rectángulos Azules («MongoDB Collection»):** Colecciones raíz de primer nivel en MongoDB con su propia clave primaria '_id: ObjectId' (db_table: users, categories, suppliers, products, orders, payments).
- **Rectángulos Verdes («Embedded Document»):** Subdocumentos incrustados directamente dentro del documento padre que no poseen colección propia (Direccion, PrecioEscalonado, DetalleVenta).
- **Rectángulo Ámbar («Inventory»):** Colección de almacén y stock físico desacoplada deliberadamente del catálogo general para alta concurrencia.

- ➔ **Flecha Azul Sólida:** Relación por referencia (vía clave foránea / ObjectId). Indica que un documento almacena la referencia del otro.
- ◆--➔ **Rombo Verde y Línea Punteada:** Composición de MongoDB (Incrustación / Embedding). El rombo se ubica en el documento contenedor y la flecha apunta al subdocumento o array embebido.

🔗 Análisis de Relaciones: Cardinalidad y Mecanismo NoSQL

En un sistema híbrido Django + MongoDB, las relaciones pueden materializarse como campos referenciados (ObjectId / ForeignKey / OneToOneField) o como subdocumentos embebidos (EmbeddedModelField / EmbeddedModelArrayField). A continuación se detalla la justificación técnica de cada relación del sistema.

1. Relaciones Uno a Uno (1:1)

- **A. auth.User ↔ Usuario (1:1 Referenciado - OneToOneField)**Extensión del modelo de autenticación estándar de Django. 'auth.User' almacena las credenciales de sesión, tokens y flags administrativos ('is_staff', 'is_superuser'). El modelo 'Usuario' actúa como perfil de dominio de negocio en la colección 'users'. Permite compatibilidad total con el Admin de Django mientras el perfil reside en una colección personalizada de MongoDB.
- **B. Usuario ↔ Direccion (1:1 Embebido doble - EmbeddedModelField)**El modelo 'Usuario' contiene dos subdocumentos embebidos independientes: 'direccion_envio' y 'direccion_facturacion'. Al guardarse en la colección 'users', no se crean registros en otra tabla. Cuando el usuario inicia sesión o carga su panel, sus direcciones se leen de forma instantánea sin requerir consultas cruzadas.
- **C. Producto ↔ Inventario (1:1 Referenciado - OneToOneField / ObjectId)**Cada 'Producto' del catálogo tiene exactamente un registro en la colección 'inventory' referenciado por 'producto_id'. ¿Por qué no embeber el inventario dentro del producto? Porque en un e-commerce y punto de venta concurrido, miles de usuarios leen el catálogo (lectura pesada), mientras los cajeros y la pasarela bloquean y actualizan cantidades de stock (escritura pesada). Mantener 'inventory' separado previene la contención de escrituras y bloqueos a nivel de documento sobre los productos.
- **D. Orden ↔ Direccion (1:1 Embebido Instantánea - EmbeddedModelField)**La colección 'orders' incluye 'direccion_entrega' como un subdocumento embebido. Esto constituye una instantánea (snapshot): si el cliente cambia su dirección de residencia semanas después en su perfil, la orden previamente despachada mantiene congelada la dirección exacta de despacho de aquel momento histórico.

2. Relaciones Uno a Muchos (1:N)

- **A. Categoria ↔ Categoria (1:N Jerarquía Autorreferencial - ForeignKey 'self')**Permite construir un árbol infinito de clasificación para semiconductores y mecatrónica (ejemplo: 'Semiconductores > Microcontroladores > 32-Bits > ARM Cortex-M4 > STM32'). Cada categoría hija apunta a su padre mediante 'parent_id'. El método 'ruta_jerarquica()' resuelve la ruta completa.
- **B. Categoria ↔ Producto (1:N Referenciado - ForeignKey / ObjectId)**Una categoría agrupa a muchos productos del catálogo, pero cada producto pertenece a una categoría principal. Al consultar productos por categoría, MongoDB indexa 'categoria_id' para filtrados ultra rápidos.
- **C. Proveedor ↔ Producto (1:N Referenciado - ForeignKey / ObjectId)**Un fabricante o proveedor (como Texas Instruments o STMicroelectronics) suministra cientos de componentes. La colección 'suppliers' guarda sus datos corporativos y líneas de especialidad, mientras los productos guardan la referencia 'proveedor_id'.
- **D. Producto ↔ PrecioEscalonado (1:N Embebido en Array - EmbeddedModelArrayField)**En el mercado de semiconductores, los componentes tienen precios por escala de volumen (Tiers: 1-9 uds a $120.00, 10-49 uds a $105.00, 50+ uds a $88.00). Estos rangos se almacenan como un Array embebido

'precios_escalonados' dentro del documento del producto. Cero JOINs: el punto de venta (POS) y el carrito calculan el precio aplicable en memoria instantáneamente.

• **E. Usuario ↔ Orden (1:N Referenciado - ForeignKey / ObjectId)**Un cliente puede generar cientos de órdenes a lo largo del tiempo. Las órdenes no se embeben dentro del usuario porque superarían el límite de 16 MB por documento BSON de MongoDB (antipatrón de arrays sin límite). Por ello, se normaliza: cada 'Orden' contiene la referencia 'usuario_id'.

• **F. Orden ↔ DetalleVenta (1:N Embebido en Array - EmbeddedModelArrayField)**Cada 'Orden' contiene el array embebido 'items' con subdocumentos 'DetalleVenta'. Cada elemento almacena una instantánea completa (producto_id, SKU, nombre, encapsulado, cantidad comprada, precio unitario aplicado y subtotal). Si el catálogo de productos cambia de precio o se descataloga en el futuro, las órdenes históricas permanecen intactas e inmutables.

• **G. Orden ↔ Pago (1:N Referenciado - ForeignKey / ObjectId)**Una orden puede recibir múltiples intentos de pago (un intento fallido de Stripe, una confirmación posterior por transferencia SPEI, o reembolsos parciales). Por tanto, la colección 'payments' guarda cada transacción vinculada a 'orden_id'.

Matriz Resumen de Relaciones y Cardinalidad

Entidad Origen	Entidad Destino	Cardinalidad	Mecanismo MongoDB	Propósito Arquitectónico
auth.User	Usuario	1 : 1	Referenciado (OneToOne)	Separar credenciales de auth del perfil de negocio
Usuario	Direccion	1 : 1 (doble)	Embebido (EmbeddedField)	Direcciones de envío y facturación sin JOINs
Producto	Inventario	1 : 1	Referenciado (OneToOne)	Desacoplar escrituras de stock de lecturas de catálogo
Orden	Direccion	1 : 1	Embebido (EmbeddedField)	Instantánea (snapshot) histórica de entrega
Categoria	Categoria	1 : N	Referenciado (FK self)	Árbol jerárquico recursivo de componentes
Categoria	Producto	1 : N	Referenciado (ForeignKey)	Clasificación de catálogo por categoría
Proveedor	Producto	1 : N	Referenciado (ForeignKey)	Asociación de fabricante a productos
Producto	PrecioEscalonado	1 : N	Embebido (ArrayField)	Precios por volumen (Tiers) dentro del producto
Usuario	Orden	1 : N	Referenciado (ForeignKey)	Historial de pedidos sin límite de 16MB
Orden	DetalleVenta	1 : N	Embebido (ArrayField)	Instantánea inmutable de productos comprados

Entidad Origen	Entidad Destino	Cardinalidad	Mecanismo MongoDB	Propósito Arquitectónico
Orden	Pago	1 : N	Referenciado (ForeignKey)	Múltiples pasarelas, reintentos y conciliaciones

📚 Diccionario Detallado de Colecciones y Documentos

A continuación se desglosan en detalle cada una de las 7 colecciones raíz y los 3 subdocumentos embebidos implementados en el código de 'miapp/models.py', detallando sus campos, tipos BSON/Django, restricciones de negocio y ejemplos en formato JSON/BSON.

1. Colección 'users' — Modelo: Usuario [Colección Raíz]

Almacena la información de perfiles, datos de contacto, rol en el sistema y las direcciones embebidas. Se relaciona 1:1 con la cuenta de autenticación de Django ('auth.User').

Campo	Tipo BSON	Restricción	Descripción y Regla de Negocio
`_id`	ObjectId	Obligatorio (PK)	Identificador único nativo generado automáticamente por MongoDB.
`user_id`	ObjectId	Opcional (1:1)	Clave foránea hacia 'auth.User' para sincronizar login y permisos.
`username`	String	Obligatorio (Unique)	Identificador alfanumérico único para login y menciones.
`email`	String	Obligatorio (Unique)	Correo electrónico verificado para notificaciones y facturación.
`nombre_completo`	String	Obligatorio (max 200)	Nombre y apellidos completos del cliente o colaborador.
`telefono`	String	Opcional (max 30)	Número de contacto telefónico para logística y paquetería.
`rol`	String (Enum)	Default: 'cliente'	Roles: 'admin', 'cajero', 'cliente', 'distribuidor', 'ingeniero'.
`direccion_envio`	Object (Embedded)	Opcional	Subdocumento embebido 'Direccion' para paquetería.
`direccion_facturacion`	Object (Embedded)	Opcional	Subdocumento embebido 'Direccion' para emisión fiscal.
`activo`	Boolean	Default: True	Flag de control para habilitar o suspender cuentas de usuario.
`creado_en`	ISODate	AutoNowAdd	Marca de tiempo de registro del usuario en la plataforma.

Ejemplo BSON: Documento en colección 'users'

```
{
```

```
  "_id": ObjectId("65c1a8e9f1a2b3c4d5e6f701"),
  "username": "roberto_maker",
  "email": "roberto@mecatronica.org",
  "nombre_completo": "Roberto Gómez Bolaños",
  "telefono": "+52 55 9876 5432",
  "rol": "ingeniero",
  "direccion_envio": {
    "calle": "Av. Insurgentes Sur 1602, Piso 4",
    "colonia": "Crédito Constructor",
    "ciudad": "Benito Juárez",
    "estado": "Ciudad de México",
    "codigo_postal": "03940",
    "pais": "México",
    "referencias": "Junto al laboratorio de robótica"
  },
  "activo": true,
  "creado_en": ISODate("2026-03-15T10:30:00Z")
}
```

2. Colección 'categories' — Modelo: Categoria [Colección Raíz]

Clasificación jerárquica en árbol recursivo para organizar componentes electrónicos, sensores y actuadores mecatrónicos.

Campo	Tipo BSON	Restricción	Descripción y Regla de Negocio
_id	ObjectId	Obligatorio (PK)	Identificador único de la categoría.
nombre	String	Obligatorio (max 150)	Nombre visible de la categoría (ej: 'Microcontroladores ARM').
slug	String	Obligatorio (Unique)	Identificador amigable para URLs indexables (ej: 'arm-cortex-m4').
descripcion	String	Opcional (Text)	Descripción técnica de la familia de componentes agrupados.
parent_id	ObjectId	Opcional (FK self)	Referencia a la categoría padre superior para armar el árbol.
activa	Boolean	Default: True	Visibilidad pública en el menú y filtros de búsqueda del catálogo.

Ejemplo BSON: Documento en colección 'categories' (Subcategoría jerárquica)

```
{
  "_id": ObjectId("65c1a8e9f1a2b3c4d5e6f710"),
  "nombre": "STM32 Cortex-M4",
  "slug": "stm32-cortex-m4",
  "descripcion": "Microcontroladores de alto rendimiento con DSP y FPU",
  "parent_id": ObjectId("65c1a8e9f1a2b3c4d5e6f709"),
  "activa": true
}
```

3. Colección 'suppliers' — Modelo: Proveedor [Colección Anexa]

Directorio de fabricantes internacionales de semiconductores, distribuidores oficiales y marcas mecatrónicas.

Campo	Tipo BSON	Restricción	Descripción y Regla de Negocio
`_id`	ObjectId	Obligatorio (PK)	Identificador único del proveedor o fabricante.
`nombre`	String	Obligatorio (max 200)	Razón social / marca (ej: 'Texas Instruments', 'STMicroelectronics').
`codigo`	String	Obligatorio (Unique)	Siglas internacionales del fabricante (ej: 'TI', 'ST', 'NXP', 'MCHP').
`sitio_web`	String (URL)	Opcional	Enlace al portal oficial corporativo del fabricante.
`email_contacto`	String	Opcional	Correo de atención para compras mayoristas y garantías.
`telefono`	String	Opcional	Teléfono de contacto o soporte de fábrica.
`pais`	String	Default: 'México'	País de origen o matriz del fabricante.
`lineas_componentes`	String	Opcional (Text)	Líneas de especialidad: amplificadores, DSP, drivers de motor, etc.
`activo`	Boolean	Default: True	Estado del proveedor para compras y reabastecimiento.
`creado_en`	ISODate	AutoNowAdd	Fecha de alta del proveedor en el sistema.

4. Colección 'products' — Modelo: Producto [Colección Raíz]

Catálogo integral de componentes con especificaciones técnicas flexibles en formato JSON nativo de MongoDB y array embebido de precios escalonados por volumen.

Campo	Tipo BSON	Restricción	Descripción y Regla de Negocio
`_id`	ObjectId	Obligatorio (PK)	Identificador único de producto en la base de datos.
`sku`	String	Obligatorio (Unique)	Número de Parte del Fabricante (MPN) (ej: 'STM32F401RET6').
`nombre`	String	Obligatorio (max 250)	Título comercial descriptivo del componente mecatrónico.
`categoria_id`	ObjectId	Obligatorio (FK)	Referencia foránea a la colección 'categories'.

Campo	Tipo BSON	Restricción	Descripción y Regla de Negocio
proveedor_id	ObjectId	Opcional (FK)	Referencia foránea al fabricante en la colección 'suppliers'.
precio_base	Decimal128	Obligatorio (12, 2)	Precio unitario para compras al por menor (1 a N unidades).
descripcion	String	Opcional (Text)	Resumen de características, arquitectura interna y periféricos.
datasheet_url	String (URL)	Opcional	Enlace oficial a la hoja de datos en formato PDF.
encapsulado	String	Opcional (max 100)	Package físico del integrado (ej: 'LQFP-64', 'SOIC-8', 'TO-220').
voltaje_operacion	String	Opcional (max 100)	Rango de alimentación eléctrica admitido (ej: '1.7V - 3.6V').
pines	Integer	Opcional	Número total de pines / leads del encapsulado.
especificaciones_tecnicas	Object (JSON)	Default: {}	Atributos dinámicos (frecuencia, flash, RAM, torque, etc.).
precios_escalonados	Array[Object]	Default: []	Lista embebida de subdocumentos 'PrecioEscalonado' (Tiers).
activo	Boolean	Default: True	Flag de publicación activa en el catálogo público.
creado_en / actualizado_en	ISODate	AutoNow	Auditoría de alta y última modificación técnica.

Ejemplo BSON: Documento 'products' con JSON flexible y array embebido de tiers

```
{
  "_id": ObjectId("65c1a8e9f1a2b3c4d5e6f720"),
  "sku": "STM32F401RET6",
  "nombre": "MCU ARM Cortex-M4 84MHz 512KB Flash",
  "categoria_id": ObjectId("65c1a8e9f1a2b3c4d5e6f710"),
  "proveedor_id": ObjectId("65c1a8e9f1a2b3c4d5e6f715"),
  "precio_base": NumberDecimal("125.50"),
  "encapsulado": "LQFP-64",
  "voltaje_operacion": "1.7V - 3.6V",
  "pines": 64,
  "especificaciones_tecnicas": {
    "frecuencia_max": "84 MHz",
    "memoria_flash": "512 KB",
    "sram": "96 KB",
    "adc_canales": 16,
    "temporizadores": 8
  },
  "precios_escalonados": [
```

```
    { "cantidad_minima": 10, "precio_unitario": NumberDecimal("112.00") },
    { "cantidad_minima": 50, "precio_unitario": NumberDecimal("98.50") },
    { "cantidad_minima": 250, "precio_unitario": NumberDecimal("84.00") }
  ],
  "activo": true
}
```

5. Colección 'inventory' — Modelo: Inventario [Colección Desacoplada 1:1]

Control físico de almacén, ubicación en estanterías/gavetas y gestión atómica de existencias. Desacoplada del catálogo de productos para soportar alta concurrencia de ventas sin generar bloqueos en el catálogo.

Campo	Tipo BSON	Restricción	Descripción y Regla de Negocio
`_id`	ObjectId	Obligatorio (PK)	Identificador único del registro de inventario.
`producto_id`	ObjectId	Obligatorio (1:1 Unique)	Referencia única obligatoria hacia el producto correspondiente.
`stock_disponible`	Integer	Default: 0	Existencias libres para venta inmediata en tienda física o web.
`stock_reservado`	Integer	Default: 0	Unidades apartadas en carritos de checkout en proceso de pago.
`ubicacion_anaquel`	String	Opcional (max 100)	Identificador físico de anaquel/gaveta (ej: 'Pasillo B, Gaveta 14-C').
`almacen`	String	Default: 'Almacén Central'	Nombre de la nave o sucursal donde reside la mercancía.
`stock_minimo_alerta`	Integer	Default: 10	Umbral crítico para disparar órdenes automáticas de reorden.
`actualizado_en`	ISODate	AutoNow	Marca de tiempo de la última operación de inventario.

Métodos Atómicos de Inventario: Lógica de Negocio y Métodos Implementados:

- • **reservar(cantidad):** Resta de 'stock_disponible' y suma a 'stock_reservado' de forma atómica para evitar sobreventa durante el proceso de pago.
- • **liberar_reserva(cantidad):** Devuelve las unidades al stock disponible si el checkout expira o el usuario vacía su carrito.
- • **confirmar_venta(cantidad):** Liquida definitivamente las unidades reservadas al completarse exitosamente la orden de pago.
- • **descontar_venta_directa(cantidad):** Descuenta directamente del stock disponible para cobros en caja física (POS) en tiempo real.

6. Colección 'orders' — Modelo: Orden [Colección Raíz]

Representa la transacción comercial consolidada. Incluye el subdocumento de dirección de entrega y el array embebido 'items' que almacena la instantánea inmutable de los productos comprados.

Campo	Tipo BSON	Restricción	Descripción y Regla de Negocio
_id	ObjectId	Obligatorio (PK)	Identificador único de la orden.
folio	String	Obligatorio (Unique)	Código único visible de venta (ej: 'ORD-2026-004821').
usuario_id	ObjectId	Obligatorio (FK)	Referencia al cliente en la colección 'users'.
estado	String (Enum)	Default: 'pendiente'	Estados: 'pendiente', 'pagada', 'preparando', 'enviada', 'entregada', 'cancelada'.
items	Array[Object]	Obligatorio	Array de subdocumentos 'DetalleVenta' congelados al momento de compra.
subtotal	Decimal128	Obligatorio	Suma de los subtotales de todos los items comprados.
costo_envio	Decimal128	Default: 0.00	Tarifa de logística o mensajería calculada.
total	Decimal128	Obligatorio	Monto neto a pagar (subtotal + costo_envio - descuentos).
direccion_entrega	Object (Embedded)	Opcional	Instantánea de la dirección física donde se entregará el pedido.
notas	String	Opcional (Text)	Instrucciones especiales de entrega o advertencias de manejo.
creado_en / actualizado_en	ISODate	AutoNow	Auditoría de generación y cambio de estado del pedido.

Ejemplo BSON: Documento 'orders' con items embebidos y totales consolidados

```
{
  "_id": ObjectId("65c1a8e9f1a2b3c4d5e6f750"),
  "folio": "ORD-2026-008412",
  "usuario_id": ObjectId("65c1a8e9f1a2b3c4d5e6f701"),
  "estado": "pagada",
  "items": [
    {
      "producto_id": "65c1a8e9f1a2b3c4d5e6f720",
      "sku": "STM32F401RET6",
      "nombre": "MCU ARM Cortex-M4 84MHz",
      "encapsulado": "LQFP-64",
      "cantidad": 10,
      "precio_unitario": NumberDecimal("112.00"),
      "subtotal": NumberDecimal("1120.00")
    },
```

```
    {
      "producto_id": "65c1a8e9f1a2b3c4d5e6f725",
      "sku": "L298N-MOD",
      "nombre": "Módulo Driver Puente H Dual L298N",
      "encapsulado": "Módulo PCB",
      "cantidad": 2,
      "precio_unitario": NumberDecimal("65.00"),
      "subtotal": NumberDecimal("130.00")
    }
  ],
  "subtotal": NumberDecimal("1250.00"),
  "costo_envio": NumberDecimal("150.00"),
  "total": NumberDecimal("1400.00"),
  "direccion_entrega": {
    "calle": "Av. Insurgentes Sur 1602",
    "ciudad": "Benito Juárez",
    "estado": "Ciudad de México",
    "codigo_postal": "03940",
    "pais": "México"
  },
  "creado_en": ISODate("2026-03-20T14:45:00Z")
}
```

7. Colección 'payments' — Modelo: Pago [Colección Anexa]

Bitácora y conciliación de pasarelas de pago externas (Stripe, PayPal, SPEI, Terminal bancaria POS).

Campo	Tipo BSON	Restricción	Descripción y Regla de Negocio
_id	ObjectId	Obligatorio (PK)	Identificador único de la transacción.
orden_id	ObjectId	Obligatorio (FK)	Referencia a la orden en la colección 'orders'.
pasarela	String (Enum)	Obligatorio	Pasarelas: 'stripe', 'paypal', 'transferencia', 'tarjeta'.
transaccion_id	String	Obligatorio (Unique)	Token, Charge ID o referencia bancaria (ej: 'ch_3N4y...').
monto	Decimal128	Obligatorio	Monto exacto cobrado a través de la pasarela.
moneda	String	Default: 'MXN'	Moneda de la transacción (MXN, USD, EUR).
estado	String (Enum)	Default: 'pendiente'	Estados: 'pendiente', 'aprobado', 'rechazado', 'reembolsado'.
datos_conciliacion	Object (JSON)	Default: {}	Respuesta cruda en JSON devuelta por la API de la pasarela.
fecha_transaccion	ISODate	AutoNowAdd	Marca de tiempo del intento o confirmación del pago.

8. Subdocumentos Embebidos (EmbeddedModel de MongoDB) [Subdocumentos]

Los siguientes modelos heredan de 'EmbeddedModel' de 'django-mongodb-backend'. No poseen colección propia ni clave primaria '_id' independiente; se incrustan como objetos BSON o arrays dentro de sus documentos contenedores.

A. Subdocumento: Direccion

Almacena la estructura estándar de localización geográfica. Se embebe 1:1 en 'Usuario' (envío y facturación) y en 'Orden' (entrega).

Campo	Tipo BSON	Restricción	Descripción
`calle`	String	Obligatorio (max 255)	Nombre de la vialidad, número exterior e interior.
`colonia`	String	Opcional (max 150)	Asentamiento o colonia de la demarcación.
`ciudad`	String	Obligatorio (max 100)	Ciudad o municipio de residencia.
`estado`	String	Obligatorio (max 100)	Entidad federativa o provincia.
`codigo_postal`	String	Obligatorio (max 20)	Código postal oficial.
`pais`	String	Default: 'México'	País de destino.
`referencias`	String	Opcional (max 255)	Instrucciones de entrega para el mensajero.

B. Subdocumento: PrecioEscalonado

Permite estructurar escalas de precios por volumen. Se embebe 1:N como array en 'Producto.precios_escalonados'.

Campo	Tipo BSON	Restricción	Descripción
`cantidad_minima`	PositiveInteger	Obligatorio	Umbral mínimo de piezas para activar la tarifa.
`precio_unitario`	Decimal128	Obligatorio	Precio con descuento por unidad a partir de dicha cantidad.

C. Subdocumento: DetalleVenta

Instantánea (snapshot) de compra congelada. Se embebe 1:N como array en 'Orden.items'.

Campo	Tipo BSON	Restricción	Descripción
`producto_id`	String	Obligatorio	ID original del producto en el catálogo.
`sku`	String	Obligatorio	Número de parte del fabricante congelado al

Campo	Tipo BSON	Restricción	Descripción
			comprar.
nombre	String	Obligatorio	Nombre comercial del componente en la fecha de compra.
encapsulado	String	Opcional	Package físico del componente vendido.
cantidad	PositiveInteger	Default: 1	Número de piezas adquiridas en el renglón.
precio_unitario	Decimal128	Obligatorio	Precio unitario pactado al momento de compra.
subtotal	Decimal128	Obligatorio	Total del renglón (cantidad × precio_unitario).

🚀 Beneficios Arquitectónicos y Consideraciones de Rendimiento

---

La arquitectura implementada combina las mejores prácticas del diseño de esquemas en MongoDB con la robustez del framework web Django 6.1:

- **1. Cero JOINs en Consultas Críticas:** Al consultar un producto para renderizar su ficha técnica o calcular su precio en el punto de venta, los precios escalonados (tiers) vienen incrustados en el mismo documento. Igualmente, al consultar una orden para imprimir el ticket o consultar el historial, todo el detalle de items se recupera en una sola lectura de disco.
- **2. Inmutabilidad y Auditoría Fiscal Garantizada:** El uso de subdocumentos 'DetalleVenta' en lugar de claves foráneas directas al catálogo de productos garantiza que si un administrador altera el precio de un microcontrolador mañana, las órdenes cerradas en el pasado conserven inalterados sus montos originales, protegiendo la consistencia contable.
- **3. Máxima Escalabilidad y Baja Contención de Bloqueos:** Al separar la colección 'inventory' del catálogo de productos, las operaciones concurrentes de cajeros y checkout no compiten por bloqueos de escritura con las miles de lecturas simultáneas del catálogo web.
- **4. Indexación Optimizada:** Campos críticos como 'username', 'email', 'slug', 'codigo', 'sku', 'folio' y 'transaccion_id' cuentan con índices únicos en MongoDB, asegurando búsquedas en tiempo logarítmico O(log N) e integridad referencial.